

Задачи с динамично програмиране

Дизайн и анализ на алгоритми

Теми: Линейно DP, DP върху мрежа

Лесни задачи (Easy)

Climbing Stairs

Условие

Дадено е стълбище с n стъпала. Можете да се качвате по **1** или **2** стъпала наведнъж. По колко различни начина можете да достигнете върха?

Вход

Едно цяло число n

Изход

Едно цяло число — броят на различните начини за изкачване.

Сложност

$O(n)$ време, $O(1)$ памет.

Примери

1. $n = 2 \Rightarrow 2$ (Два начина: $\{1, 1\}$ или $\{2\}$.)
2. $n = 3 \Rightarrow 3$ (Три начина: $\{1, 1, 1\}$, $\{1, 2\}$ или $\{2, 1\}$.)

Насока за решение

Нека $dp[i]$ е броят начини да стигнем до стъпало i . Последното движение е или от стъпало $i - 1$ (1 стъпало нагоре), или от стъпало $i - 2$ (2 стъпала нагоре), откъдето следва рекурентността $dp[i] = dp[i - 1] + dp[i - 2]$, с базови случаи $dp[1] = 1$, $dp[2] = 2$. Забележете, че това е точно редицата на Фибоначи. Обосновете защо е достатъчно да пазим само последните две стойности (оптимизация на паметта до $O(1)$).

Min Cost Climbing Stairs

Условие

Дадена е редица $cost$ с n елемента, където $cost[i]$ е цената за стъпване на стъпало i . След заплащане на цената можете да се качите с **1** или **2** стъпала. Можете да тръгнете от стъпало 0 или стъпало 1. Намерете минималната цена за достигане на стъпалото *след* последното (индекс n).

Вход

Масив $cost[1..n]$

Изход

Едно цяло число — минималната цена.

Сложност

$O(n)$ време, $O(1)$ памет.

Примери

1. $\text{cost} = [10, 15, 20] \Rightarrow 15$ (Тръгваме от стъпало 1, плащаме 15, скачаме 2 напред.)
2. $\text{cost} = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1] \Rightarrow 6$

Насока за решение

Дефинирайте $dp[i]$ като минималната цена за достигане на стъпало i . Рекурентността е $dp[i] = \text{cost}[i] + \min(dp[i-1], dp[i-2])$, с базови случаи $dp[0] = \text{cost}[0]$, $dp[1] = \text{cost}[1]$. Отговорът е $\min(dp[n-1], dp[n-2])$. Обосновете коректността и покажете как паметта се редуцира до $O(1)$ чрез пазене само на последните две стойности.

Divisor Game

Условие

Алиса и Боб играят игра с цяло число n . На всеки ход текущият играч избира цяло число x , такова че $1 \leq x < n$ и $n \bmod x = 0$ (т.е. x е делител на n), и замества n с $n - x$. Играчът, който не може да направи ход (когато $n = 1$), губи. Алиса е на ход първа. Върнете **true**, ако Алиса печели при оптимална игра и от двамата.

Вход

Едно цяло число n .

Изход

true ако Алиса печели; **false** в противен случай.

Сложност

$O(n^2)$ (или $O(1)$ при наблюдение на закономерност).

Примери

1. $n = 2 \Rightarrow \text{true}$ (Алиса избира $x = 1$, n става 1, Боб губи.)
2. $n = 3 \Rightarrow \text{false}$ (Алиса е принудена да избере $x = 1$, n става 2, Боб печели.)

Насока за решение

Дефинирайте $dp[i] = \text{true}$, ако текущият играч печели при $n = i$. Базов случай: $dp[1] = \text{false}$. За всяко $i \geq 2$: $dp[i] = \text{true}$, ако съществува делител x на i , такъв че $dp[i-x] = \text{false}$ (т.е. можем да поставим опонента в губеща позиция). Забележете закономерността: $dp[i] = \text{true}$ тогава и само тогава, когато i е четно. Обосновете математически тази закономерност и анализирайте сложността на двата подхода.

Средна задача (Medium)

Unique Paths

Условие

Робот се намира в горния ляв ъгъл на правоъгълна мрежа с размери $m \times n$. Роботът може да се движи само **надясно** или **надолу** на всяка стъпка. Целта е да достигне долния десен ъгъл на мрежата. По колко различни пътя може да стигне до целта?

Вход

Две цели числа m и n .

Изход

Едно цяло число — броят на различните пътища.

Сложност

$O(m \cdot n)$ време, $O(m \cdot n)$ памет (или $O(n)$ с оптимизация).

Примери

1. $m = 3, n = 7 \Rightarrow 28$
2. $m = 3, n = 2 \Rightarrow 3$ (Три пътя: ДДН, ДНД, НДД, където Д = надясно, Н = надолу.)

Насока за решение

Нека $dp[i][j]$ е броят пътища до клетка (i, j) . Тъй като роботът може да дойде само отляво или отгоре, рекурентността е $dp[i][j] = dp[i-1][j] + dp[i][j-1]$, с базови случаи $dp[0][j] = 1$ и $dp[i][0] = 1$ (по ръбовете има точно по един път). Обосновете коректността на рекурентността и покажете как паметта може да се оптимизира до $O(n)$, като поддържате само текущия ред. Алтернативно, отговорът е комбинаторен: $\binom{m+n-2}{m-1}$ — обосновете и тази формула.

За всяка задача се очаква: описание на алгоритъма стъпка по стъпка, доказателство за коректност и анализ на времева сложност

Контакти

При въпроси, свързани с тези или други задачи, може да се свържете с мен тук:

Instagram: [instagram.com/stanislav._igov](https://www.instagram.com/stanislav._igov)

Facebook: [facebook.com/stanislav.igov](https://www.facebook.com/stanislav.igov)